RESEARCH PROPOSAL

Department of Informatics / Computer Science

IMPLEMENTATION OF DIGITAL IMAGE STEGANOGRAPHY USING THE LEAST SIGNIFICANT BIT (LSB) METHOD FOR PYTHON-BASED SECRET MESSAGE HIDING

Submitted to:

[Lecturer / Course Name]

Prepared by:

[Author Name]

[Student ID] | [Institution]

[City], [Year]

Contents

I. Background

The rapid advancement of information and communication technology in the current digital era has brought fundamental changes to the way humans exchange information. The internet, as a global communication medium, enables instant transmission of large volumes of data across geographical boundaries. However, this convenience has also given rise to various threats to information security and privacy. Data transmitted over public networks is vulnerable to eavesdropping, theft, and manipulation by irresponsible parties. Therefore, the need for reliable information security techniques has become increasingly critical in preserving data confidentiality and integrity.

Steganography is one information security technique that has developed significantly in the digital era. Unlike cryptography, which encodes message content to render it unreadable, steganography works by concealing the very existence of a message within a carrier medium (cover medium), allowing covert communication to take place without arousing the suspicion of third parties. Common carrier media include digital images, audio files, video, and text documents. This approach makes steganography an effective complement to cryptography in building layered information security systems (Cheddad et al., 2010). In the context of modern cybersecurity, the combination of both techniques provides more comprehensive protection than using either method alone.

The Least Significant Bit (LSB) method is the most fundamental steganography technique and is widely studied in the domain of digital image processing. Its working principle is based on modifying the least significant bit of each pixel value in an image, so that the resulting changes cannot be visually detected by the human eye. According to Hussain et al. (2018), LSB offers high message embedding capacity with minimal image quality degradation, making it the primary choice in spatial-domain steganography implementations. Furthermore, implementing LSB with the Python programming language and the PIL/Pillow and NumPy libraries provides ease of development along with flexibility in pixel data manipulation. Thus, this research aims to comprehensively implement, evaluate, and analyze an LSB-based steganography system in order to understand its characteristics, capacity, and limitations in real-world usage scenarios.

II. Problem Statement

Based on the background described above, the research problems addressed in this study are as follows:

1. How can a Least Significant Bit (LSB)-based steganography system be designed and implemented using the Python programming language with the PIL/Pillow and NumPy libraries to hide text messages within PNG-format digital images?

2. What is the maximum capacity of text messages that can be embedded in a digital image without causing visually detectable image quality degradation, as measured using the Peak Signal-to-Noise Ratio (PSNR) and Mean Squared Error (MSE) metrics?

3. What is the robustness of the implemented LSB steganography system against image compression and file format transformation processes, and to what extent can LSB value histogram distribution analysis reveal the presence of hidden messages (steganalysis)?

III. Research Objectives

3.1 General Objective

This research generally aims to implement a digital image steganography system based on the Least Significant Bit (LSB) method using Python, while evaluating its performance through standardized image quality parameters.

3.2 Specific Objectives

More specifically, this research aims to:

1. Design and build an encoder module capable of embedding text messages into digital images (PNG format) using Python-based LSB manipulation techniques with a user-friendly interface.

2. Design and build a decoder module capable of accurately and completely extracting hidden messages from stego-images without data loss.

3. Measure and analyze the quality of the resulting stego-images using the objective metrics PSNR and MSE, and compare the results against threshold values established in the scientific literature.

4. Determine the maximum message embedding capacity in pixels per bit for various input image sizes as a practical usage guide for the system.

5. Conduct a robustness analysis of the system against JPEG compression attacks and histogram-based LSB distribution steganalysis.

IV. Significance of Research

4.1 Theoretical Significance

1. Provide a deep understanding of the working mechanism of LSB steganography in the spatial domain of digital images, including the characteristics of bit-level manipulation and its impact on image quality.

2. Enrich the scientific literature on stego-image quality measurement through the implementation and comparative analysis of PSNR and MSE metrics in the context of steganography.

3. Provide a comprehensive academic reference on Python-based steganography implementation as a foundation for further study in the fields of information security and image processing.

4.2 Practical Significance

1. Produce a functional steganography software prototype that can be used by information security practitioners for legitimate covert communication purposes.

2. Provide a technical implementation guide for steganography systems that can serve as a reference for developing information security systems in both academic and industrial settings.

3. Raise awareness of the importance of steganography as an additional layer of digital communication security in an era of increasing cyber threats.

V. Scope and Limitations

To maintain focus and analytical depth, this research limits its scope to the following:

1. The input image format is limited to PNG (Portable Network Graphics) files, which support lossless compression, to ensure pixel data integrity after the message embedding process.

2. The embedded message medium is limited to UTF-8 encoded text only, and does not include embedding of binary files, images, or other media.

3. The steganography technique implemented is limited to the one-bit LSB method on the RGB (Red, Green, Blue) color channels, without considering embedding on the alpha channel or single-channel (grayscale) images.

4. The programming language used is Python 3.x with the latest version of the PIL/Pillow library and NumPy, without involving artificial intelligence or machine learning frameworks for the steganalysis process.

5. Image quality evaluation is limited to computational measurement of the PSNR and MSE metrics, and does not include perceptual quality assessment using methods such as SSIM.

6. The research does not cover the development of a message encryption (cryptography) system prior to embedding; the focus is entirely on the steganography aspect without additional cryptographic security layers.

VI. Literature Review

6.1 Definition of Steganography

The term steganography originates from the Greek words steganos, meaning "hidden," and graphein, meaning "to write," and can therefore be literally translated as "hidden writing." In the context of computer science and information security, steganography is defined as the art and science of concealing secret information within another data medium (cover medium) in such a way that the existence of that information cannot be detected by unauthorized parties (Cox et al., 2007). The fundamental difference between steganography and cryptography lies in their security approaches: cryptography conceals message content through encryption, whereas steganography conceals the very existence of the message (Gonzalez & Woods, 2018).

In a digital steganography system, there are three main components: (1) the cover object, the medium used as the message carrier; (2) the secret message, the information to be concealed; and (3) the stego object, the cover object that has been embedded with the secret message. The quality of a steganography system is generally evaluated based on three criteria: capacity, imperceptibility (undetectable by human perception), and robustness against steganalysis attacks.

6.2 Brief History of Steganography

The concept of steganography has been known for thousands of years before the digital era. In ancient Greece, Herodotus recorded the use of a technique involving writing messages on the shaved heads of slaves, then allowing the hair to grow back before sending the slave as a message carrier. During World War II, invisible ink and microdot techniques were widely used for military intelligence purposes (Wayner,

2009).

In the digital era, steganography evolved rapidly alongside the growth of multimedia technology. This development began with fundamental research on data embedding in the spatial domain of digital images in the 1990s, progressing to sophisticated frequency-domain techniques such as the Discrete Cosine Transform (DCT) and Discrete Wavelet Transform (DWT) developed in subsequent decades. Marvel et al. (1999) introduced the concept of spread spectrum image steganography, which became an important foundation for the development of steganography techniques more resistant to steganalysis attacks.

6.3 Digital Image Processing

Digital image processing is a field of study that examines computational image manipulation techniques to produce meaningful information or improve image quality (Gonzalez & Woods, 2018). A digital image is represented as a two-dimensional (or three-dimensional for color images) matrix composed of the smallest elements called pixels (picture elements).

In a grayscale image, each pixel is represented by a single intensity value in the range of 0 to 255 (8 bits). In a color image using the RGB model, each pixel consists of three color channels – Red (R), Green (G), and Blue (B) – each with a bit depth of 8, giving a total of 24 bits per pixel. This binary representation forms the basis of the bit manipulation technique in LSB steganography. By altering the least significant bits of pixel values, the resulting change in intensity is extremely small (at most 1 out of 256 levels), making it visually undetectable.

6.4 The Least Significant Bit (LSB) Method

The Least Significant Bit (LSB) is a spatial-domain steganography method that works by replacing the least significant bit of an image's pixel values with the bits that make up the secret message (Hussain et al., 2018). As an illustration, the pixel value 200 in 8-bit binary representation is 11001000. If the LSB (rightmost position) is replaced with the message bit "1," the value becomes 11001001, or 201 – a change that is virtually indistinguishable visually.

Technically, the LSB encoding process involves the following steps: (1) converting the text message into binary representation; (2) iterating through each pixel of the image; (3) replacing the LSB of each pixel with the corresponding message bit; and (4) saving the result as a stego-image. The decoding process is the reverse: (1) extracting the LSB from each pixel of the stego-image; (2) concatenating the extracted bits; (3) converting the combined bits into ASCII/Unicode characters;

and (4) reading the message until a delimiter is detected.

The message embedding capacity using the 1-bit LSB method on an RGB image is calculated using the formula: C = (W
texttimes H
texttimes 3) / 8 bytes, where W is the image width and H is the image height in pixels. Thus, an 800
texttimes600-pixel image can theoretically hold up to 180,000 bytes, or approximately 175 KB of text characters.

6.5 Comparison of Steganography Methods

In addition to LSB, several other steganography methods are commonly studied in the scientific literature. The Discrete Cosine Transform (DCT) operates in the frequency domain of an image and is commonly used with JPEG format. The DCT method offers better robustness against compression but has a lower embedding capacity compared to LSB (Kaur & Kumar, 2020). The Discrete Wavelet Transform (DWT) uses wavelet transformation to embed messages in the frequency sub-bands of an image, providing a better balance between capacity, image quality, and attack robustness.

A comparison of these three methods shows that LSB excels in implementation simplicity and embedding capacity, but is relatively vulnerable to steganalysis attacks based on statistical LSB analysis. Conversely, DCT and DWT are more resistant to steganalysis but require more complex computation. In the context of this research, the choice of the LSB method is based on considerations of ease of implementation, high capacity, and strong educational value as a foundation for understanding steganography.

6.6 PNG vs. JPEG Image Formats in Steganography

The choice of image format is a critical factor in steganography implementation. The JPEG (Joint Photographic Experts Group) format uses a lossy DCT-based compression algorithm that actively modifies pixel values during the saving process, meaning that data embedded using the LSB method will be corrupted after the image is saved in JPEG format. In contrast, the PNG (Portable Network Graphics) format uses a lossless compression algorithm (DEFLATE) that preserves pixel values intact without modification, making it the ideal choice for LSB steganography implementation (Gonzalez & Woods, 2018).

The BMP (Bitmap) format also supports pixel storage without lossy compression, but produces very large file sizes compared to PNG. Therefore, PNG is the most

recommended format for LSB steganography implementation due to its combination of lossless compression, 24/32-bit color support, and more manageable file sizes compared to BMP.

VII. Research Methodology

7.1 Research Type

This research uses a computational experimental research approach, in which the steganography system is designed, implemented, and empirically evaluated through standardized quality metric measurements. The research paradigm employed is applied research, oriented toward the development and evaluation of a functional software system.

7.2 Tools and Materials

7.2.1 Hardware

1. Computer with a minimum Intel Core i3 processor or equivalent (i5/i7 recommended for high-resolution image processing)
2. Minimum 4 GB RAM (8 GB recommended)
3. SSD/HDD storage with a minimum capacity of 10 GB for the test image dataset

7.2.2 Software

1. Operating System: Windows 10/11, Linux Ubuntu 20.04+, or macOS 11+
2. Python 3.x (Python Software Foundation, 2024) as the primary programming language
3. PIL/Pillow (Clark et al., 2024): image processing library for pixel manipulation
4. NumPy: numerical computing library for matrix operations and bit array manipulation
5. Matplotlib: data visualization library for LSB distribution histogram analysis
6. Jupyter Notebook or VS Code as the Integrated Development Environment (IDE)
7. Git for source code version management

7.3 Research Stages

This research is carried out through the following systematic stages:

Literature Review

At this stage, an in-depth review of relevant scientific literature is conducted, covering digital image processing textbooks (Gonzalez & Woods, 2018), recent steganography journals, and technical documentation for the Python libraries used. The literature review aims to build a strong theoretical foundation before the system design stage.

Encoder System Design

The encoder module design covers: (1) defining the input data structure (image path, message string); (2) designing the algorithm for converting the message to binary; (3) designing the per-pixel bit embedding mechanism using LSB; and (4) designing a delimiter system to mark the end of the hidden message.

Decoder System Design

The decoder module design covers: (1) a mechanism for sequentially reading pixels from the stego-image; (2) extracting the LSB per pixel channel; (3) reconstructing characters from the extracted bits; and (4) detecting the delimiter to stop the extraction process and return the complete message.

Python Code Implementation

Implementation is carried out based on the designed specifications using Python 3.x with PIL/Pillow for image manipulation and NumPy for array operations. The source code is organized into separate modules (encoder.py, decoder.py, utils.py, metrics.py) with complete documentation following the PEP 8 standard.

Testing and Evaluation

Testing is performed using a dataset of test images of various resolutions and content types (nature photography, human faces, textures). Evaluation covers PSNR and MSE measurements, embedding capacity analysis, and histogram distribution analysis of the LSB values before and after message embedding.

7.4 Encoding and Decoding Process Flowchart

The encoding process flow starts from: (1) INPUT: Cover image (PNG) + text message; (2) Convert message to 8-bit binary representation per character + delimiter; (3) Read image pixels sequentially (R, G, B); (4) Replace pixel LSB with message bit; (5) Save result as stego-image (PNG); (6) OUTPUT: Stego-image containing

the hidden message.

The decoding process flow starts from: (1) INPUT: Stego-image (PNG); (2) Read pixels sequentially; (3) Extract LSB from each pixel channel (R, G, B); (4) Concatenate bits into bytes (8 bits); (5) Convert bytes to characters; (6) Detect end-of-message delimiter; (7) OUTPUT: Extracted text message.

VIII. System Design

8.1 Encoder System Architecture

The encoder module accepts two primary inputs: a PNG-format digital image as the cover image and a text message string to be concealed. The system then converts the text message into binary representation using UTF-8 encoding, where each character is represented as 8 bits (for standard ASCII characters). A special binary delimiter (a unique bit sequence such as "0000000000000000" representing a null character) is appended at the end of the message to mark the end of the data.

The embedding process is carried out by iterating through image pixels row by row, accessing the R, G, and B channel values in turn. For each pixel, the LSB of the color channel's integer value (0-255) is replaced with the corresponding message bit. The bitwise operation used is: new_value = (pixel_value & 0xFE) | message_bit, where 0xFE (11111110 in binary) is used as a mask to ensure that only the LSB is modified. The resulting embedded image is then saved in PNG format using the save() method from PIL/Pillow.

8.2 Decoder System Architecture

The decoder module accepts the stego-image as a single input. The system reads the image pixels sequentially in the same order as the encoding process (row by row, R-G-B channels). The LSB from each pixel value is extracted using the bitwise operation: bit = pixel_value & 0x01 (AND operation with 1). The extracted bits are then accumulated and grouped in sets of 8 bits to form a single character.

The reconstructed characters are stored in a string buffer. The system continuously checks for the presence of the end-of-message delimiter with each new character formed. When the delimiter is detected, the extraction process is stopped and the string buffer (without the delimiter) is returned as the message output. This approach ensures computational efficiency by halting the process as soon as the entire message has been successfully extracted.

8.3 Technical Explanation of Bit Manipulation

The core operation in this system is LSB manipulation at the level of individual pixel bytes. In an 8-bit binary number system, bit 0 (the rightmost bit) has a positional value of 2^0 = 1, meaning that modifying this bit changes the pixel value by at most 1 unit within the range of 0-255. This change is equivalent to a color intensity change of less than 0.4%, well below the visual detection threshold of the human visual system, which generally requires an intensity difference of at least 1-2% to be perceptible.

The theoretical system capacity is calculated based on the image dimensions: an RGB image with a resolution of W
texttimes H pixels can hold a maximum of (W
texttimes H
texttimes 3) message bits, equivalent to (W
texttimes H
texttimes 3) / 8 byte characters. In practice, the effective capacity is lower because some pixels are used to store header information (message length) and the delimiter. The uniform distribution of pixels across the entire image area makes the stego-image visually indistinguishable from the original cover image.

IX. Testing and Evaluation Plan

9.1 Image Quality Evaluation Metrics

9.1.1 Peak Signal-to-Noise Ratio (PSNR)

PSNR is the industry-standard metric for measuring the reconstruction quality of a digital image, expressed in decibels (dB). The PSNR formula is: $PSNR = 10 \times \log_{10}(MAX^2/MSE)$, where MAX is the maximum pixel value (255 for an 8-bit image) and MSE is the Mean Squared Error. A higher PSNR value indicates better stego-image quality. In the context of steganography, a PSNR value above 40 dB is generally considered an acceptable quality threshold, indicating that the difference between the cover image and the stego-image is not visually detectable.

9.1.2 Mean Squared Error (MSE)

MSE measures the average squared difference in pixel values between the original cover image and the produced stego-image. The MSE formula is: $MSE = (1/(M \times N)) \times \sum(I(i,j) - K(i,j))^2$, where I is the cover image, K is the stego-image, and M texttimes N is the image dimension. An MSE value approaching zero indicates that the stego-image is nearly identical to the cover image. MSE and PSNR have an

inversely proportional relationship, making both metrics complementary in quality evaluation.

9.2 Storage Capacity Testing

Capacity testing is performed by embedding messages of varying lengths (100, 500, 1,000, 5,000, and 10,000 characters) into test images of various resolutions (512
texttimes512, 1024
texttimes768, and 1920
texttimes1080 pixels). The test results will be analyzed to determine the optimal capacity ratio that yields PSNR values above 40 dB.

9.3 Visual Testing and LSB Histogram Analysis

LSB bit value histogram distribution analysis is performed to compare the bit distribution patterns in the cover image and the stego-image. In a natural cover image, the LSB distribution tends to follow a normal distribution with a relatively balanced frequency between values 0 and 1. After message embedding, this distribution may change statistically, forming the basis of steganalysis techniques. Histogram visualizations using Matplotlib will be generated for both conditions to identify any statistical pattern changes that may occur.

9.4 Robustness Testing against Compression

System robustness is tested by saving the stego-image in JPEG format at various quality levels (25%, 50%, 75%, and 100%), then attempting to decode the message from the compressed image. This test aims to confirm the hypothesis that the LSB method is not resistant to lossy JPEG compression, and to provide quantitative data on message degradation at various compression levels. These results will serve as a recommendation for the exclusive use of PNG format in LSB steganography implementations.

X. References

Books

Cheddad, A., Condell, J., Curran, K., & Mc Kevitt, P. (2010). Digital image steganography: Survey and analysis of current methods. Signal Processing, 90(3), 727-752. https://doi.org/10.1016/j.sigpro.2009.08.010

Cox, I., Miller, M., Bloom, J., Fridrich, J., & Kalker, T. (2007). Digital watermarking and steganography (2nd ed.). Morgan Kaufmann.

Gonzalez, R. C., & Woods, R. E. (2018). Digital image processing (4th ed.). Pearson.

Wayner, P. (2009). Disappearing cryptography: Information hiding: Steganography & watermarking (3rd ed.). Morgan Kaufmann.

Journal Articles

Hussain, M., Wahab, A. W. A., Idris, Y. I. B., Ho, A. T. S., & Jung, K. H. (2018). Image steganography in spatial domain: A survey. Signal Processing: Image Communication, 65, 46-66. https://doi.org/10.1016/j.image.2018.03.012

Kaur, M., & Kumar, V. (2020). A comprehensive review of image steganography techniques. International Journal of Information Security, 19(1), 1-26. https://doi.org/10.1007/s10207-018-0426-0

Marvel, L. M., Boncelet, C. G., & Retter, C. T. (1999). Spread spectrum image steganography. IEEE Transactions on Image Processing, 8(8), 1075-1083. https://doi.org/10.1109/83.777088

Subhedar, M. S., & Mankar, V. H. (2014). Current status and key issues in image steganography: A survey. Computer Science Review, 13-14, 95-113. https://doi.org/10.1016/j.cosrev.2014.09.001

Pevny, T., Filler, T., & Bas, P. (2010). Using high-dimensional image models to perform highly undetectable steganography. Information Hiding, 6387, 161-177. https://doi.org/10.1007/978-3-642-16435-4_13

Fridrich, J., Goljan, M., & Du, R. (2001). Reliable detection of LSB steganography in color and grayscale images. Proceedings of the 2001 Workshop on Multimedia and Security: New Challenges, 27-30. https://doi.org/10.1145/1232454.1232466

Online Sources and Technical Documentation

Clark, A., et al. (2024). Pillow (PIL Fork) documentation. Pillow. https://pillow.readthedocs.io

OpenCV Team. (2024). OpenCV documentation. OpenCV. https://docs.opencv.org

Python Software Foundation. (2024). Python 3.x documentation. https://docs.python.org

NumPy Community. (2024). NumPy documentation. https://numpy.org/doc/

Matplotlib Development Team. (2024). Matplotlib documentation. https://matp

lotlib.org/stable/

This proposal is prepared as an academic document for research in the fields of Information Security and Digital Image Processing.